Análisis del software: vistas dinámicas de Lava Rápido Vehicular

Santiago Gordo Pérez

José Stiven Perdomo Martínez

Servicio Nacional de Aprendizaje (SENA)

Tecnólogo en Análisis y Desarrollo de Software

Ficha 3145555

Instructor: José de Jesús Motta Vargas

4 de noviembre de 2026

Resumen

El presente documento analiza el comportamiento del sistema Lava Rápido Vehicular y las interacciones entre la aplicación móvil, el panel web administrativo, la API y la base de datos. Describe casos de uso, secuencias, actividades y transiciones de estados para la reserva, el pago y la atención de servicios. El análisis se fundamenta en la especificación de requisitos del proyecto y en la estructura y reglas identificadas en los repositorios disponibles al 4 de noviembre de 2026. Se diferencia la funcionalidad documentada en código de las verificaciones que requieren ejecución integrada. El producto se organiza mediante una arquitectura cliente-servidor con backend monolítico modular y PostgreSQL. Las vistas dinámicas muestran que el cobro mediante Wompi necesita verificación desde el backend y que el estado operativo de una reserva es independiente del estado de pago.

Palabras clave: análisis de software, UML, diagramas de secuencia, reservas, pasarela de pagos, arquitectura de software.

Contenido

1. Introducción
2. Contexto, alcance y actores
3. Arquitectura y componentes
4. Casos de uso
5. Diagramas de secuencia
6. Diagramas de actividades
7. Diagramas de estados
8. Persistencia y reglas de negocio
9. Seguridad y calidad
10. Trazabilidad y limitaciones
11. Conclusiones
Referencias

1. Introducción

Lava Rápido Vehicular es una plataforma para organizar la prestación de servicios de lavado de vehículos. Permite que los clientes consulten servicios, registren vehículos, creen reservas, paguen y sigan la atención; que los operadores gestionen sus asignaciones; y que los administradores mantengan el catálogo y supervisen la operación (Equipo de desarrollo de Lava Rápido Vehicular, 2026a).

1.1 Objetivo general

Analizar la estructura funcional y el comportamiento del sistema mediante vistas dinámicas basadas en sus componentes actuales, para documentar interacciones, decisiones, restricciones y estados que intervienen en los procesos de negocio.

1.2 Objetivos específicos

• Identificar actores, módulos y responsabilidades en los canales móvil y administrativo.

• Representar los casos de uso y los flujos de reserva, cobro y ejecución mediante diagramas.

• Especificar estados y transiciones de reservas y pagos según las reglas del backend.

• Relacionar los flujos con los servicios de aplicación y la persistencia, distinguiendo evidencia estática de pruebas ejecutadas.

2. Contexto, alcance y actores

La plataforma opera con tres perfiles principales: cliente, operador y administrador. El cliente y el operador utilizan la aplicación móvil; el administrador utiliza el panel web. Wompi participa como proveedor externo de pagos. Los servicios de notificaciones pueden incorporar Expo Push y SMTP cuando el entorno está configurado para ello.

Actor	**Responsabilidades identificadas**
Cliente	Autenticarse, consultar catálogo, administrar vehículos, reservar, pagar, cancelar cuando corresponda, consultar

	avisos y calificar.
Operador	Consultar asignaciones propias y registrar inicio o finalización de la atención.
Administrador	Administrar servicios, marcas y operadores, consultar reservas, asignar operadores y gestionar pagos autorizados.
Wompi	Procesar transacciones y suministrar estado remoto o eventos verificables.

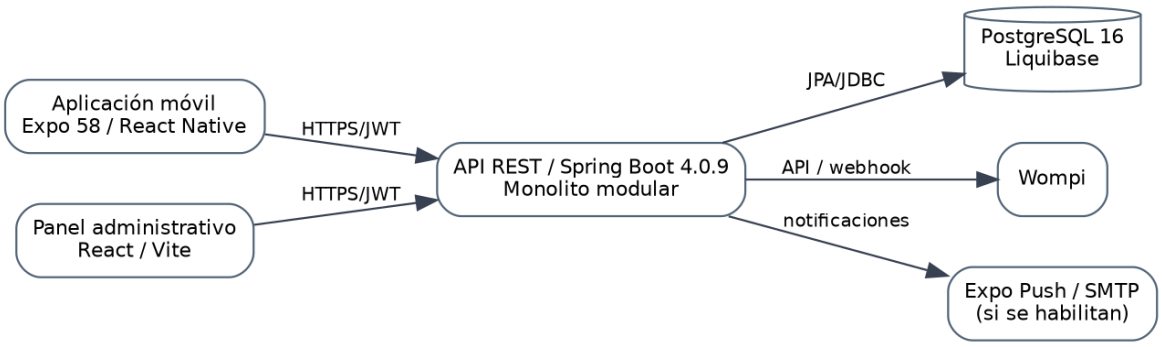
Quedan fuera del alcance el seguimiento GPS real de operarios, la optimización de rutas, el inventario físico y la contabilidad detallada. Tampoco se presume la existencia de un módulo independiente de administración de usuarios/roles o de reasignación explícita cuando no se ha identificado en los clientes disponibles.

3. Arquitectura y componentes

El sistema utiliza una arquitectura cliente-servidor. La API Spring Boot está organizada como monolito modular, con separación por dominios y responsabilidades Controller, Service, Repository, Entity y DTO. Los clientes intercambian solicitudes HTTP, mientras que PostgreSQL conserva los datos y Liquibase administra la evolución declarada del esquema. La aplicación móvil declara Expo SDK 58 y React Native 0.88; el backend declara Java 17 y Spring Boot 4.0.9.

Figura 1

Vista general de componentes y comunicaciones



Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

Componente	Tecnologías declaradas	Responsabilidad
Móvil	Expo ~58.0.20, React Native 0.88, TypeScript	Experiencia del cliente y operador; solicitudes a API.
Web administrativa	React 19, Vite 6, TypeScript	Operación administrativa y consultas autorizadas.
Backend	Spring Boot 4.0.9, Java 17, Spring Security, JPA	Reglas de negocio, control de acceso e integraciones.
Persistencia	PostgreSQL 16, Liquibase	Datos relacionales, restricciones y migraciones.

3.1 Módulos funcionales

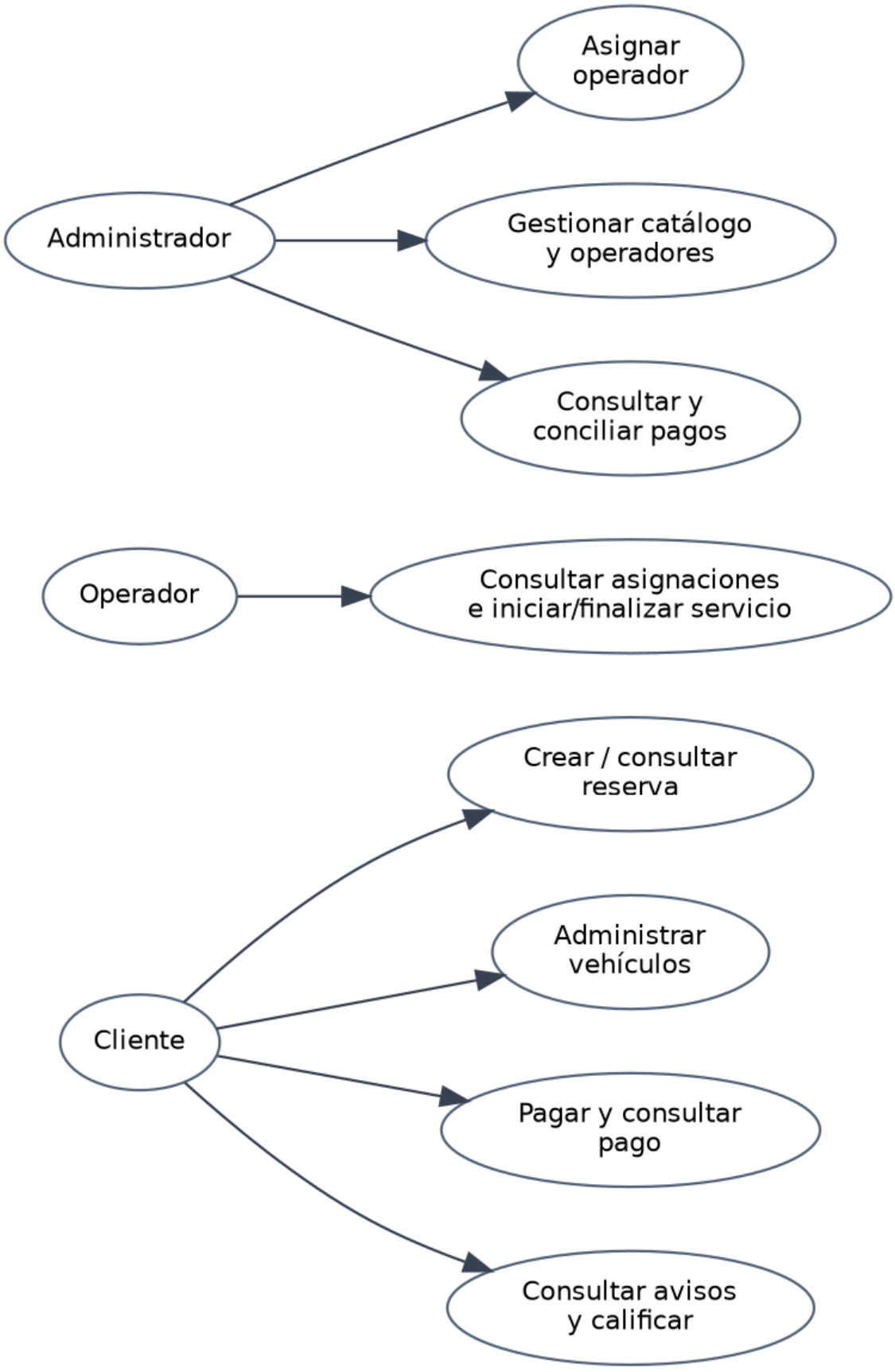
En el backend se identifican módulos de autenticación, usuarios, servicios, marcas, vehículos, reservas, operadores, asignaciones, pagos, calificaciones, notificaciones, auditoría, errores y salud. La API implementa los controles de propiedad y autorización; la visibilidad de una opción en la interfaz no sustituye esos controles.

4. Casos de uso

Un caso de uso describe el objetivo de un actor y la respuesta esperada del sistema. La Figura 2 resume las capacidades identificadas; los casos de pagos incluyen la participación de un proveedor externo, sin considerar la devolución del widget como aprobación definitiva.

Figura 2

Casos de uso principales por actor


Asignar operador
Administrador
Gestionar catálogo y operadores
Consultar y conciliar pagos
Operador
Consultar asignaciones e iniciar/finalizar servicio
Crear / consultar reserva
Administrar vehículos
Cliente
Pagar y consultar pago
Consultar avisos y calificar

Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

Código	Caso de uso	Precondición y resultado
CU-01	Crear reserva	Cliente autenticado, vehículo propio activo, servicio disponible; genera reserva PENDIENTE.
CU-02	Iniciar y verificar pago	Reserva apta; genera/reutiliza intento y consulta la confirmación verificable.
CU-03	Asignar operador	ADMIN y operador activo; genera asignación única por reserva.
CU-04	Atender servicio	Operador autorizado; registra estados operativos permitidos.
CU-05	Cancelar reserva	Titular y condiciones de estado/pago; cancelación persistida.
CU-06	Calificar servicio	Reserva finalizada y titular autorizado; máximo una calificación.

5. Diagramas de secuencia

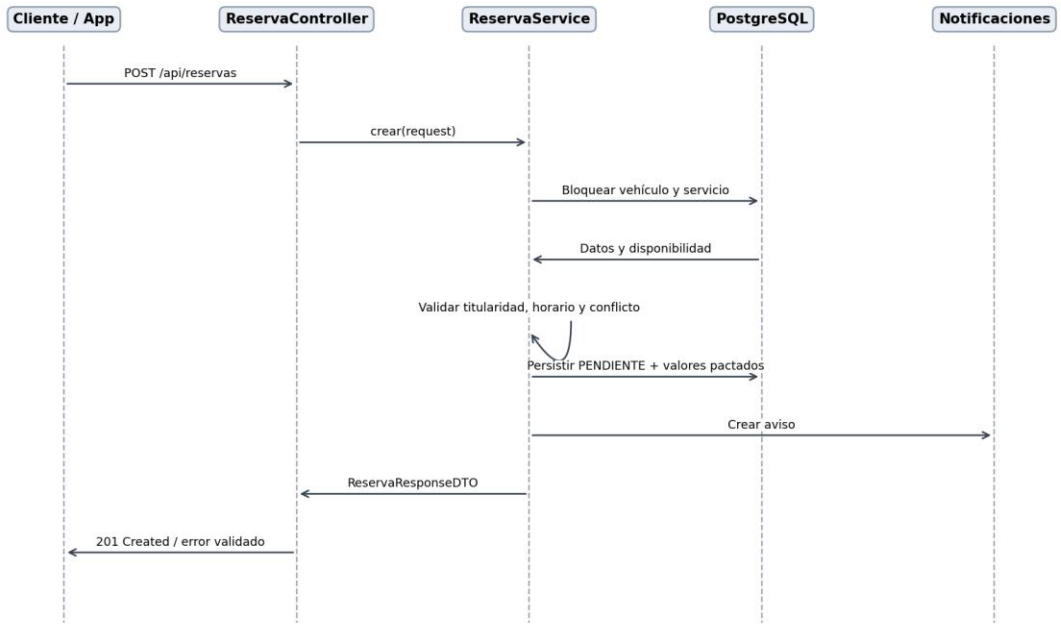
Los diagramas siguientes representan el orden lógico de mensajes entre actores y componentes. Las rutas y métodos ilustrados están respaldados por controladores y servicios de los repositorios; su inclusión no equivale a una ejecución de extremo a extremo.

5.1 Creación de reserva

El cliente envía vehículo, servicio, fecha y hora mediante POST /api/reservas. ReservaService comprueba propiedad, actividad, horario y solapamiento por vehículo, conserva precio y duración pactados y persiste la reserva PENDIENTE. La API devuelve el resultado o un error controlado.

Figura 3

Secuencia de creación de reserva



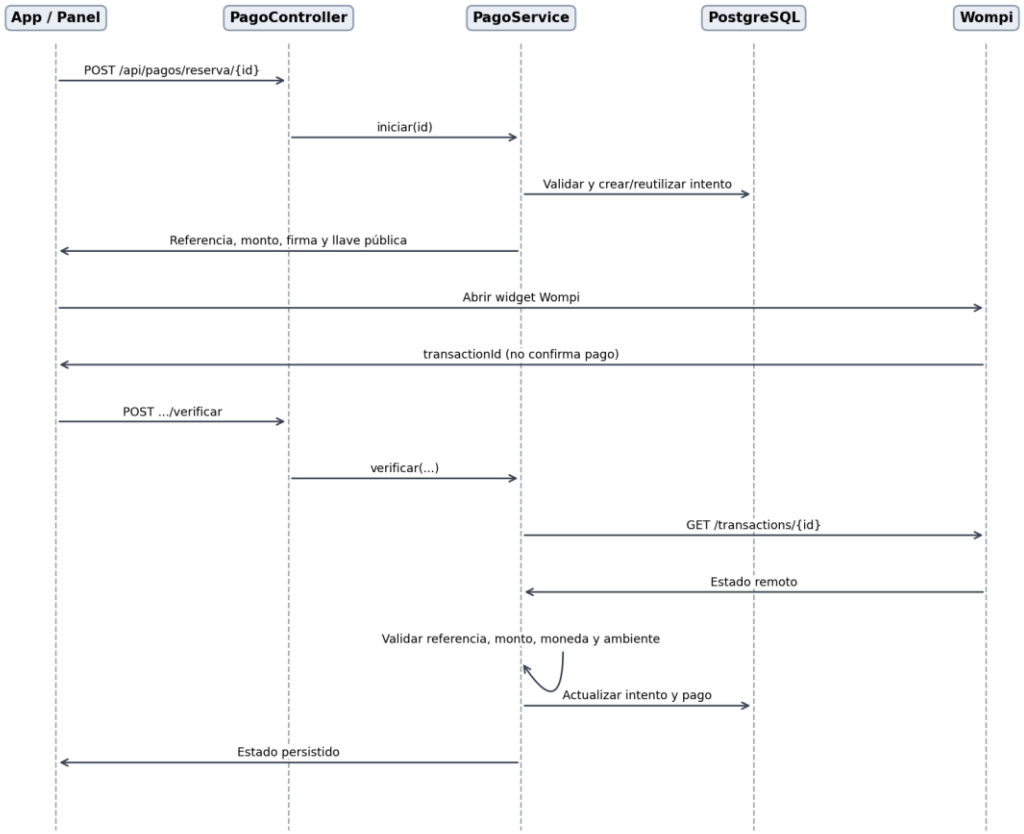
Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

5.2 Pago en línea y confirmación

El backend inicia un intento para una reserva apta y suministra datos públicos del checkout. La respuesta del widget no confirma el pago: la verificación requiere comparar la transacción remota con identificador, referencia, ambiente, monto y moneda COP. El sistema también contempla webhooks autenticados por checksum y conciliación de intentos. El estado del pago no cambia automáticamente el estado operativo de la reserva.

Figura 4

Secuencia principal de pago con Wompi



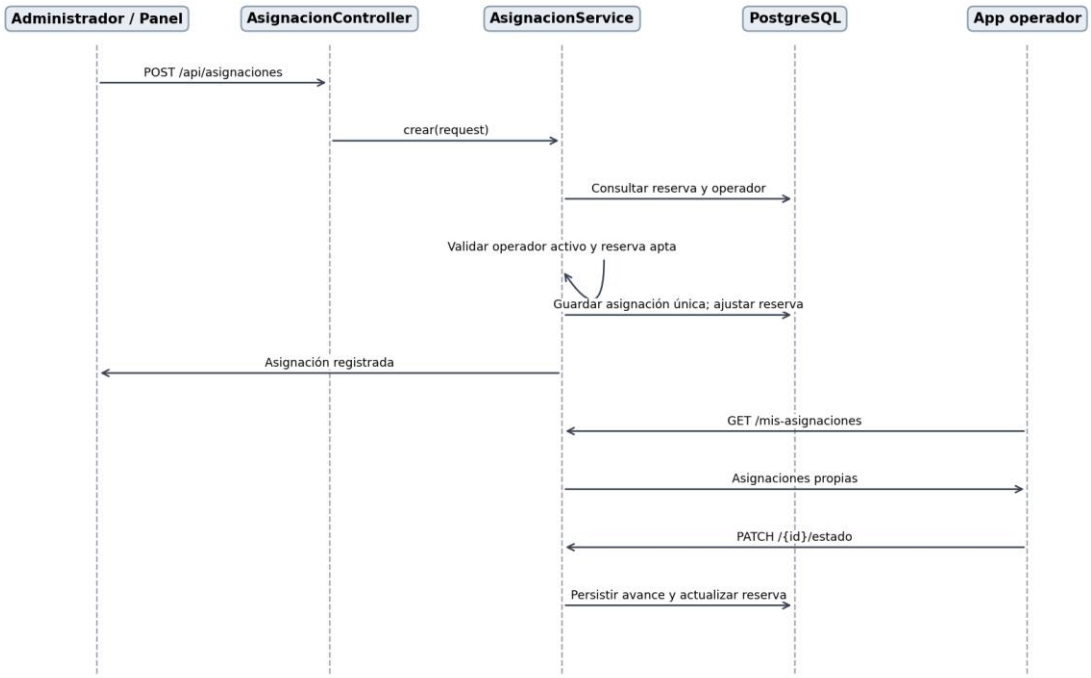
Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

5.3 Asignación y ejecución del servicio

El administrador selecciona una reserva apta y un operador activo. El servicio comprueba que no exista otra asignación para la misma reserva, registra la asignación y, según el estado, actualiza la reserva. El operador consulta únicamente sus tareas y registra el progreso mediante un endpoint protegido.

Figura 5

Secuencia de asignación y progreso del operador



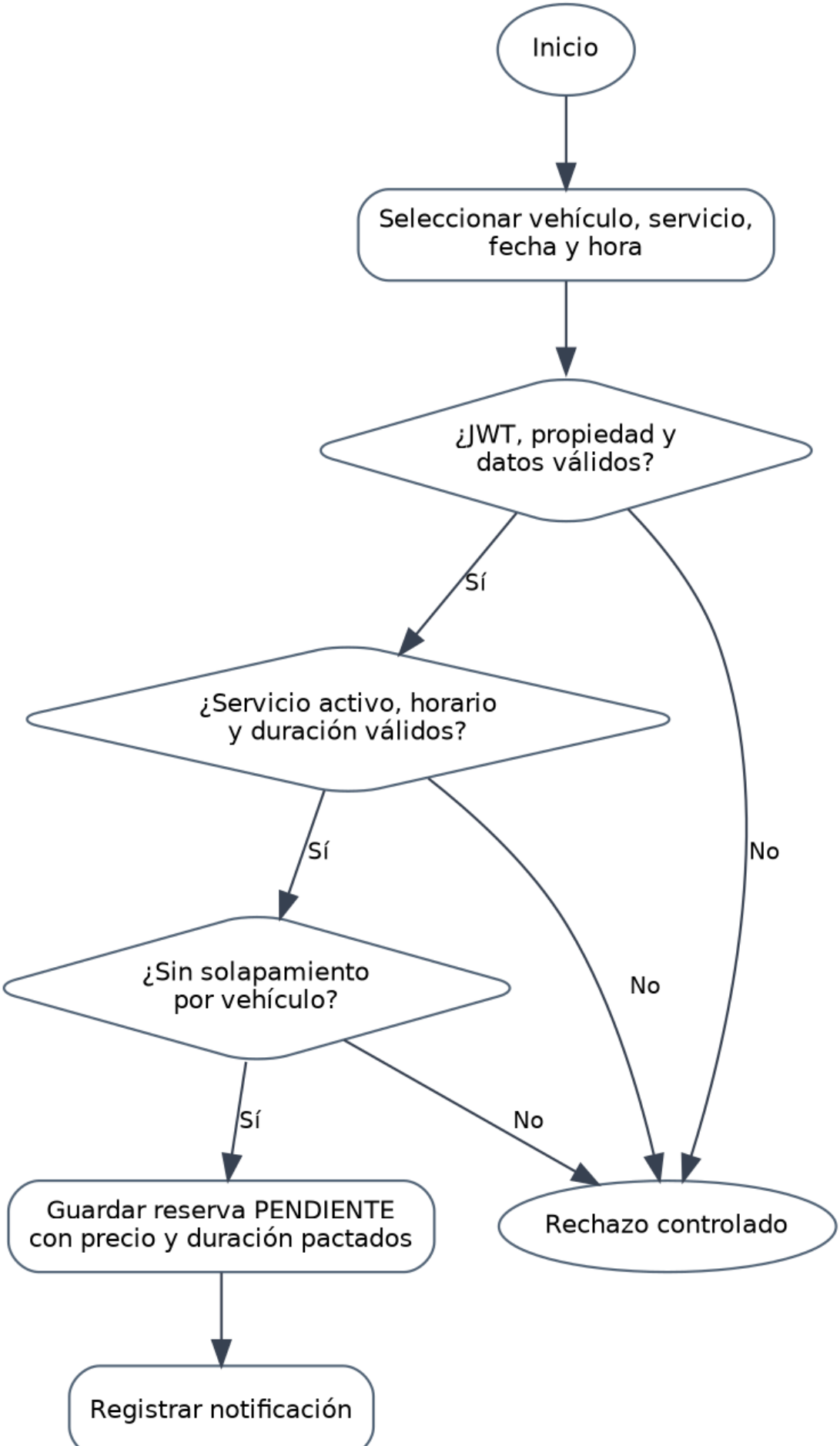
Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

6. Diagramas de actividades

El proceso de reserva contempla decisiones de seguridad y negocio antes de escribir datos. Una reserva solo se persiste cuando el vehículo pertenece a la cuenta, está habilitado, el servicio está activo, el horario resulta válido y no hay conflicto en el intervalo solicitado.

Figura 6

Actividad de creación y validación de reserva


Inicio
Seleccionar vehículo, servicio, fecha y hora
¿JWT, propiedad y datos válidos?
Sí
No
¿Servicio activo, horario y duración válidos?
Sí
No
¿Sin solapamiento por vehículo?
Sí
No
Guardar reserva PENDIENTE con precio y duración pactados
Rechazo controlado
Registrar notificación

Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

6.1 Flujos alternos

• Autenticación: los accesos sin sesión, con cuenta no autorizada o con credenciales incorrectas generan errores controlados.

• Reserva: un vehículo ajeno, inactivo, o un intervalo solapado impiden su creación.

• Pago: una firma inválida o datos de transacción inconsistentes no pueden generar aprobación.

• Conectividad: después de una interrupción, el cliente debe consultar el estado persistido antes de reintentar una operación susceptible a duplicación.

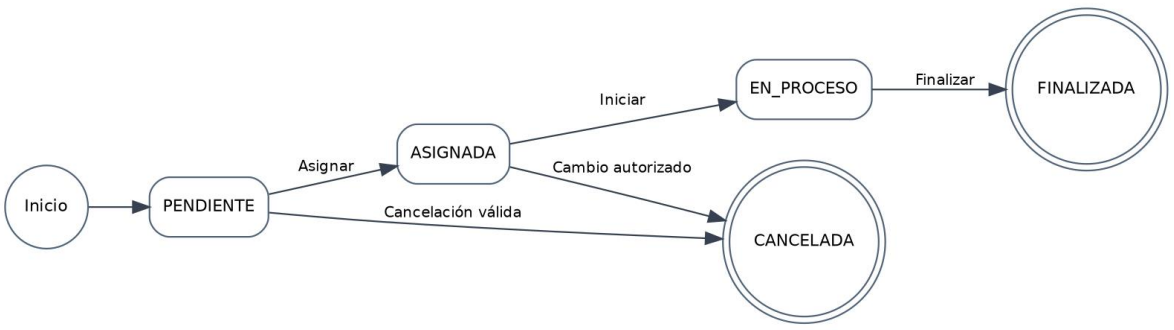
7. Diagramas de estados

7.1 Reserva

Los estados del enum EstadoReserva son PENDIENTE, ASIGNADA, EN_PROCESO, FINALIZADA y CANCELADA. La cancelación voluntaria del cliente está sujeta a reglas más restrictivas que un cambio administrativo: requiere reserva pendiente y condiciones del pago que permitan cancelar. Por ello, una flecha en el modelo describe una transición potencial cuyo permiso depende del contexto.

Figura 7

Estados y transiciones de reserva



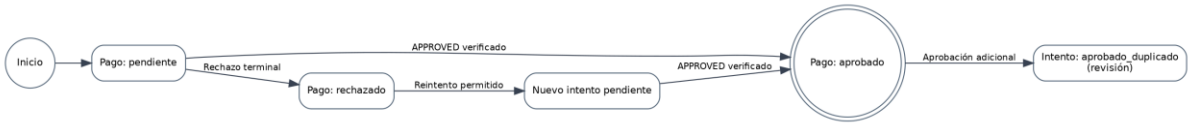
Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

7.2 Pago e intento de pago

El pago registra pendiente, aprobado o rechazado; cada intento de pago puede estar pendiente, aprobado, rechazado o aprobado_duplicado. Un rechazo terminal puede permitir un nuevo intento; una aprobación adicional se trata como caso de revisión y no como nueva aprobación de la misma reserva. Los eventos repetidos deben ser tratados sin duplicar efectos.

Figura 8

Transiciones de pago y tratamiento de duplicados



Nota. Elaboración propia a partir de los componentes y reglas identificados en los repositorios del proyecto.

8. Persistencia y reglas de negocio

En los scripts SQL revisados se declaran 18 tablas y 24 changeSets activos en Liquibase. Estas cantidades corresponden al repositorio, no certifican que todos los cambios estén aplicados en un entorno desplegado. Entre las entidades están users, roles, vehiculos, servicios, reservas, asignaciones, operadores, pagos, pago_intentos, calificaciones, notificaciones, dispositivos_push y notificacion_entregas.

Regla	Implementación identificada
Horario	Atención de reservas dentro de 06:00 a 20:00.
Integridad de reserva	Propiedad del vehículo, actividad del servicio y conservación de precio/duración pactados.
Solapamiento	Validación de negocio y restricción de exclusión GiST para el mismo vehículo.
Asignación	Una asignación por reserva; operador activo y transición permitida.
Pagos	Unicidad de referencias e intentos pertinentes; confirmación remota o evento validado.

Calificaciones	Calificación sobre reserva finalizada y máximo una por reserva.

9. Seguridad y calidad

Spring Security protege la API mediante JWT y autorización por rol y titularidad. Se identifican contraseñas protegidas con BCrypt, tokens de recuperación con controles de caducidad y un filtro de limitación de intentos de autenticación. La API exige validación para la información de pagos. El panel mantiene JWT en sessionStorage, por lo que deben conservarse medidas de prevención de XSS; el límite de intentos de autenticación se mantiene por instancia en memoria. La base declara restricciones relacionales e índices relevantes; aun así, la consistencia del esquema efectivamente desplegado y las integraciones con Wompi, SMTP o Expo Push requieren validación operativa adicional. También se identificó que el registro de auditoría persistente no cubre necesariamente todos los eventos y errores.

9.1 Estado de pruebas

Los repositorios contienen archivos de prueba en backend, aplicación móvil y panel. La información disponible contabiliza 33 archivos de pruebas Java, 58 archivos de pruebas móviles y 33 archivos del panel; estas cifras representan archivos identificados, no resultados de pruebas aprobadas. El estado actual de las suites y la operación de servicios externos no se verificaron en una ejecución integrada como parte de la información utilizada para este documento.

10. Trazabilidad y límites de verificación

Elemento	**Base técnica de referencia**
Reservas	ReservaController, ReservaService, EstadoReserva, changelog de precio y solapamiento.
Pagos	PagoController, PagoService, WompiSignatureService, WompiTransactionClientImpl.
Asignaciones	AsignacionController, AsignacionService, EstadoAsignacion.
Seguridad	SecurityConfig, JwtService, AuthRateLimitFilter.

Persistencia	changelog-master.yaml, SQL DDL y entidades JPA.
Interfaz	Servicios HTTP y pantallas de lava-rapido-mobile y lava-rapido-admin.

El alcance de esta documentación describe el estado identificado en archivos de los repositorios con fecha de corte 4 de noviembre de 2026. No se infiere la disponibilidad del despliegue, la ejecución satisfactoria de pruebas, la aplicación efectiva de migraciones ni la entrega real de notificaciones. La documentación de requerimientos expresa el comportamiento esperado y las clases/contratos revisados sustentan la descripción del comportamiento implementado.

11. Conclusiones

Lava Rápido Vehicular dispone de clientes móvil y web que interactúan con un backend monolítico modular y una base PostgreSQL. Los procesos de mayor relevancia son la reserva, la asignación y el pago: cada uno combina validaciones de identidad, reglas de negocio y cambios persistentes. Las vistas dinámicas evidencian la separación entre el estado operativo de la reserva y el estado de pago, y la necesidad de verificar las operaciones externas en el servidor. Este modelo ofrece una base trazable para el mantenimiento y la sustentación del proyecto; su validación de funcionamiento integral depende de pruebas controladas vinculadas a las versiones desplegadas.

Referencias

Equipo de desarrollo de Lava Rápido Vehicular. (2026a). Especificación de requisitos de software (SRS): Lava Rápido Vehicular [Documento interno del proyecto]. Servicio Nacional de Aprendizaje.

Equipo de desarrollo de Lava Rápido Vehicular. (2026b). Repositorios del proyecto Lava Rápido Vehicular: frontend móvil, frontend administrativo, backend y base de datos [Código fuente y configuraciones, estado al 4 de noviembre de 2026].

Object Management Group. (2017). OMG unified modeling language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Wompi. (s. f.). Documentación para desarrolladores. https://docs.wompi.co/